



Erstellung eines Datentreibers zur Verwaltung von Texten in verschiedenen Sprachen

Projektarbeit T2000

über die Praxisphase des zweiten Studienjahrs

an der Fakultät für Technik
im Studiengang Informationstechnik

an der DHBW Ravensburg
Campus Friedrichshafen

von
Eric Erath

6. September 2024

Bearbeitungszeitraum:	02. 04. 2024 - 07. 06. 2024
Matrikelnummer, Kurs:	1910897, TIT22
Dualer Partner:	fpt Systems GmbH
Betreuer des Dualen Partners:	Simon Würzer

Selbstständigkeitserklärung

gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg vom 29.09.2017.

Ich versichere hiermit, dass ich meine Projektarbeit mit dem Thema:

Erstellung eines Datentreibers zur Verwaltung von Texten in verschiedenen Sprachen

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Hergensweiler, 6. September 2024

Ort, Datum

SW-Entw.,



Abteilung, Unterschrift

Zusammenfassung

Die Software „FlexOP“ wird von der Firma fpt zur Bedienung von Roboteranlagen verwendet. Für internationale Kunden kann die Software in verschiedene Sprachen übersetzt werden. Noch nicht alle Bestandteile der Software unterstützen mehrere Sprachen.

Um weitere Teile der Software mehrsprachig zu gestalten, wird in diesem Projekt ein Datentreiber für mehrsprachige Textvariablen entworfen, implementiert und getestet. Der Treiber erhält Texte in verschiedenen Sprachen und speichert diese in einer Datenbank. Beim Zugriff auf einen Text gibt der Datentreiber die Übersetzung in der aktuell ausgewählten Sprache zurück. Der Treiber wird zudem leicht konfigurierbar sein, sodass er möglichst ideal an die Vorlieben der Kunden angepasst werden kann.

Zusätzlich wird auch ein AddIn zum Speichern von Roboterpositionen auf Mehrsprachigkeit erweitert. Die Namen der Positionen werden bereits in einer Tabelle mit verschiedenen Spalten für unterschiedliche Übersetzungen gespeichert, allerdings erhalten alle Spalten den gleichen Inhalt.

Schlüsselwörter CSharp (C#), mehrsprachig, Datentreiber, konfigurierbar, XML, Datenbank, FlexOP

Abstract

The software „FlexOP“ software is used by the company fpt to operate robot systems. The software can be translated into various languages for international customers. Not all components of the software support multiple languages yet.

In order to make further parts of the software multilingual, this project is concerned with designing, implementing and testing a data driver for multilingual text-variables. The driver receives texts in different languages and stores them in a database. When accessing a text, the data driver returns the translation in the currently selected language. The driver will also be easily configurable so that it can be ideally adjusted to the customer's preferences.

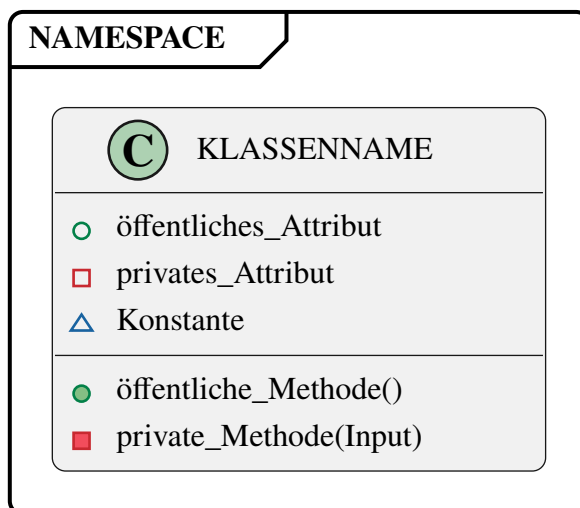
In addition, an AddIn for saving positions of the robot will also be extended to be multilingual. The Names of saved positions are already saved in a table with multiple columns for different languages, however all columns receive the same content.

Keywords CSharp (C#), multilingual, datadriver, configurable, XML, Database, FlexOP

Vorwort

In dieser Projektarbeit wird bewusst auf die Verwendung von geschlechterspezifischen Formulierungen verzichtet und das generische Maskulinum verwendet. Dies soll keinesfalls als gewollte Ausgrenzung von Gruppierungen gedeutet werden, sondern zur Lesbarkeit und Vermeidung von umständlichen Formulierungen beitragen. Weibliche und anderweitige Geschlechteridentitäten sind dabei ausdrücklich mitgemeint. Der Fokus dieser Arbeit liegt gänzlich auf der neutralen und wissenschaftlichen Behandlung des Themas, nicht auf der korrekten Einbindung aller Geschlechter.

In den Klassendiagrammen in dieser Arbeit wurden einige Attribute und Methoden zur besseren Übersichtlichkeit nicht dargestellt. Zudem werden die „Access Modifiers“ der Member von Klassen mithilfe von farbigen Symbolen dargestellt.



Inhaltsverzeichnis

Selbstständigkeitserklärung	II		
Zusammenfassung	III		
Abstract	III		
Vorwort	IV		
Abkürzungsverzeichnis	VI		
Abbildungsverzeichnis	VII		
1 Grundlagen	1		
1.1 Icon-Programme	1		
1.2 AddIns und Datentreiber . . .	1		
2 Problemstellung	2		
3 Implementationsentwurf	3		
3.1 Requirements	3		
3.2 Integration in FlexOP	4		
3.2.1 Integration im Code .	4		
3.2.2 Eintragung in die Projektdatenbank . . .	5		
3.3 XML-Konfiguration	6		
3.3.1 Parameter	7		
3.3.2 Aufteilung in mehre- re Dateien	9		
3.4 Datenbankformat	10		
3.5 Handhabung der Texte	11		
3.5.1 ErrorMode	11		
3.5.2 DeprecatedMode . . .	12		
4 Umsetzung	13		
4.1 Ex- und Importieren der AddIn-Daten	13		
4.2 Kompatibilität mit verschie- denen Dotnet-Versionen	13		
4.3 Identifikation über ID anstelle vom Namen	14		
4.4 Konflikte zwischen Zugriffs- methoden auf die Datenbank .	15		
5 Qualitätskontrolle	17		
5.1 Hilfreiche Features	17		
5.1.1 Neuladen der Konfi- guration	17		
5.1.2 Neuladen der Daten- bankeinträge	18		
5.2 Tests	18		
5.2.1 XML-Konfiguration .	18		
5.2.2 Datenbank	19		
5.2.3 Verhalten des Treibers	19		
5.2.4 Import und Export . .	20		
6 Reflexion	22		
6.1 Zusammenfassung	22		
6.2 Limitationen und mögliche Erweiterungen	22		
7 Fazit	24		
Literatur	25		
A Anhang	A		

Abkürzungsverzeichnis

fpt	„FPT Systems GmbH“	1
Icon-Prog	„fpt ICON Programming“	1
OPCUA	„Open Platform Communication Unified Architecture“	1
SPS	„Speicherprogrammierbare Steuerung“	1
SQL	„Structured Query Language“	15

Abbildungsverzeichnis

3.1	Klassendiagramm der Einbindung in FlexOP	5
3.2	Einbindung von Datentreibern in der FxBase-Datenbank	6
3.3	Ausschnitt aus der Klasse FxA_MLDBVarDriver.Config	7
3.4	Ausschnitt aus der Default-Konfiguration der Datentreivers	7
3.5	Konfigurationsparameter des MLDB-Datentreivers	8
3.6	Klassendiagramm der Datenbank-Utilities	9
3.7	Beispielinhalt der Datenbank	10
3.8	Konfigurationsparameter zur Handhabung von Texten	11
4.1	Betriebssystemprüfung für Versionen NET6 und neuere	14
5.1	Testinhalt der Datenbank	20
5.2	Erwartete Testergebnisse	20

1 Grundlagen

Die Firma „FPT Robotik GmbH & Co. KG“ spezialisiert sich auf das Entwickeln von Roboteranlagen für internationale Kunden. Die Programmierung dieser Anlagen wird von der Tochterfirma „FPT Systems GmbH“ (fpt) übernommen. Hierbei übernimmt eine „Speicherprogrammierbare Steuerung“ (SPS) die Steuerung der Peripherie der Anlage und kommuniziert mit der Robotersteuerung. Die eigens geschriebene Software FlexOP bietet eine grafische Oberfläche zur Konfiguration, Überwachung und Programmierung¹. Die Kommunikation zwischen den beiden Programmen findet hauptsächlich über den Standard „Open Platform Communication Unified Architecture“ (OPCUA) statt.

1.1 Icon-Programme

Bei der Programmierung einer Icon-Anlage wird der Arbeitsablauf in FlexOP mithilfe der grafischen Programmiersprache „fpt ICON Programming“ (Icon-Prog) festgelegt. Icon-Prog ist fpts Ansatz, mithilfe einer Programmiersprache alle Roboterarchitekturen bedienen zu können. Zudem sind Kunden, dank der grafischen Programmierart, selbst in der Lage das Programm anzupassen. „Durch einen einheitlichen visuellen Ansatz für die Konfiguration und Programmierung mehrerer Automatisierungsplattformen können die Benutzer ihre eigenen Anwendungen konfigurieren.“ (FPT Robotik GmbH 2024, S. 4) Nach der Auslieferung ist es Anwendern mit ausreichenden Berechtigungen also möglich, das Programm zu bearbeiten und eventuell geänderten Anforderungen nachzukommen.

1.2 AddIns und Datentreiber

AddIns sind optionale Erweiterungen von FlexOP. Einige AddIns werden bei nahezu jeder Roboteranlage verwendet und wurden daher in die Standardversion von FlexOP integriert. Für diese Arbeit sind die beiden Standard-AddIns „FxA_Hand“ und „FxA_ProgManager“ relevant. Das AddIn „FxA_Hand“ managt gespeicherte Positionsdaten, sogenannte Teachpositionen. Der ProgManager verwaltet die verwendeten Icon-Programme.

Datentreiber, als Spezialform von AddIns, erleichtern in FlexOP den Zugriff auf externe Daten. Beispielsweise übernimmt der OPCUA-Datentreiber die Übertragung von Variablenwerten zwischen FlexOP und der SPS.

¹ Sofern es sich um eine in Icons programmierte Anlage handelt

2 Problemstellung

Um internationalen Kunden die Bedienung der Roboteranlagen zu erleichtern, bietet die Software FlexOP die Möglichkeit in verschiedene Sprachen übersetzt zu werden. Bisher werden die Übersetzungen vor der Auslieferung der Anlage in eine Datenbank eingetragen und zur Laufzeit in der entsprechenden Sprache ausgelesen und angezeigt. Bei Textfeldern die zur Laufzeit bearbeitet werden können, bedeutet das, dass Änderungen in einer Sprache nicht korrekt in die anderen Sprachen übertragen werden können, weshalb der bisherige Ansatz hier nicht verwendet werden kann.

Hierzu soll ein neuer Datentreiber geschrieben werden, der die Möglichkeit bietet, Texte, die zur Runtime von Nutzern eingegeben werden, zusammen mit deren Übersetzungen in einer Datenbank zu verwalten. Der Treiber soll zudem über eine XML-Datei konfigurierbar sein, sodass das Verhalten an verschiedene Requirements von Kunden angepasst werden kann.

Neben der Erstellung eines neuen Datentreibers soll dieselbe Funktionalität in die AddIns FxA_Hand und FxA_ProgManager implementiert werden. Die Namen der Teachpositionen und die Beschreibungstexte der Icons können von Anwendern zur Laufzeit bearbeitet werden und demnach ebenfalls spezialisiert in der Datenbank verwaltet werden müssen. Auch hier ist die Konfigurierbarkeit via XML essenziell.

3 Implementationsentwurf

Zu Beginn des Projekts wurden von Simon Würzer, dem Betreuer des Projekts und Teamleiter der Abteilung „Software-Entwicklung“ die grundlegenden Requirements festgelegt, dass ein Datentreiber zu erstellen ist, der Texte in der Datenbank verwaltet und konfigurierbar sein soll. Darauf aufbauend wurde ein Konzept entworfen, wie die Anforderung konkret umgesetzt werden können, insbesondere welche Konfigurationsparameter für den Treiber verwendet werden. Der Entwurf wurde dann diskutiert und nach Bedarf angepasst. Im Folgenden wird die finale Version des erarbeiteten Entwurfs dargestellt.

3.1 Requirements

Um die Übersetzbarkeit von FlexOP voranzutreiben, soll ein neuer Datentreiber implementiert werden, der Textvariablen in verschiedenen Sprachen in einer Datenbank verwaltet. Der Datentreiber soll wie alle bisher bestehenden Treiber über eine XML-Datei konfigurierbar sein. Wichtige Konfigurationsparameter sind hierbei Einstellungen zur Handhabung von veralteten oder fehlenden Übersetzungen, sodass der Treiber in jedem Projekt möglichst auf die Wünsche der Kunden angepasst werden kann. Zudem soll auch die zu verwendende Datenbank und Tabelle im Treiber konfigurierbar sein. Wichtig hierbei ist, dass die Benennung der Spalten kompatibel mit dem firmeninternen Tool „Language-Manager“ ist. Der Language-Manager hilft bei der Übersetzung von FlexOP und soll daher auch für die Übersetzung der vom Datentreiber genutzten Tabelle verwendbar sein.

Um die neuen Funktionalitäten möglichst sinnvoll auch in die beiden AddIns „FxA_Hand“ und „FxA_ProgManager“, die teilweise auch Texte verwalten, übernehmen zu können, sollen hilfreiche Klassen und Methoden zur Arbeit mit mehrsprachigen Tabellen getrennt vom Datentreiber implementiert werden. Auch Methoden zum generellen Arbeiten mit Datenbanken sollen im Namespace „Fx.FlexOPFramework.Util“ öffentlich zugänglich gemacht werden. Die Implementation der allgemeinen Datenbankmethoden kann aus dem AddIn „FxFlexParam2“ von Visual Basic zu C# übersetzt werden. So wird zudem versucht zu erreichen, dass in Zukunft solche Methoden nicht mehr für jede Klasse, die mit Datenbanken arbeitet, neu implementiert werden müssen.

Große Teile von FlexOP, die bereits Mehrsprachigkeit unterstützen, sind auf sechs Sprachen limitiert, also Deutsch und fünf weitere Fremdsprachen. Den Treiber so zu implementieren, dass die Anzahl verfügbarer Sprachen nicht limitiert ist, wäre möglich. Da FlexOP allerdings vorerst nicht auf mehr Sprachen erweitert wird, der Datentreiber nur für FlexOP verwendet werden soll und bei einer Erweiterung in jedem Fall große Teile der Software geändert werden müssen, soll der neue Treiber auch nur die festgelegte Anzahl Sprachen unterstützen. Um eine potenzielle Erweiterung auf mehr als fünf Sprachen jedoch nicht unnötig zu verkomplizieren, wird die Anzahl

der verfügbaren Sprachen in der Datentreiberkonfiguration als Konstante festgelegt. So kann die Anzahl zumindest bei einer Änderung auf mehr, oder weniger, Sprachen einfach angepasst werden.

3.2 Integration in FlexOP

FlexOP ist eine Software, welche über viele Jahre gewachsen und somit recht komplex ist. Daher ist es nicht möglich in dieser Arbeit das Zusammenspiel aller Klassen darzustellen. Stattdessen werden im Folgenden nur die Schnittstellen zu den restlichen Bestandteilen beleuchtet. Das bezieht sich nicht nur auf die Infrastruktur des Codes, denn FlexOP greift auf viele verschiedene Konfigurationsdateien und Datenbanken auf dem System zu. Die genaue Funktion jeder Datei zu ergründen und erklären ist nicht das Ziel dieses Projekts.

Die Klassendiagramme in dieser Arbeit zeigen zudem nicht alle Member der Klassen an, sondern nur diejenigen, welche für die Erklärungen am relevantesten sind. Zudem ist wichtig zu wissen, dass sich vor jedem Member einer Klasse ein spezielles Symbol befindet. Methoden und Attribute mit einem grünen Kreis sind „public“. Ein rotes Quadrat bedeutet „private“ und blaue Dreiecke signalisieren Konstanten.

3.2.1 Integration im Code

In FlexOP werden Datentreiber vom VariableProvider in einer Liste veraltet. Der Provider bietet für den Rest von FlexOP eine Abstraktionsebene, die automatisch entscheidet, welche Variable zu welchem Treiber gehört. Um das zu ermöglichen, muss jeder Treiber das Interface „IFxVariableDriver“ implementieren. In diesem Interface sind alle Member definiert, die der VariableProvider zum Interagieren mit den einzelnen Datentreibern benötigt. Die wichtigste Methode ist hierbei die „GetVariable(...)“-Methode, die einige Informationen über die zu übergebende Variable erhält. Die Methode hat zudem einen optionalen Parameter, der einen Defaultwert für die Variable angibt, für den Fall, dass die Variable noch nicht existiert und neu erstellt werden muss. Wenn eine noch nicht existierende Variable angefordert wird, ohne dass ein Defaultwert angegeben ist, verwendet der neue Datentreiber einen leeren String als Default. Als Rückgabewert ist im Datentreiber-Interface eine Implementation der „IFxDriverVariable“ festgelegt. Die „FxTranslationVariable“, die vom Datentreiber verwendet wird erbt von der Variablen-Basisklasse, die wiederum das Interface implementiert.

Da alle Datentreiber in FlexOP auch AddIns sind, und gleich konfigurierbar sein sollen, erweitert der neue MLDB²-Treiber die Basisklasse „FxFlexOpAddIn“. So erbt der Treiber unter anderem eine Instanz der nötigen Konfigurationsklasse, hier „Config“, die als Erbe der „FxXmlConfiguration“ von der AddIn-Basisklasse verwendet werden kann.

²Multi-Language-DataBase

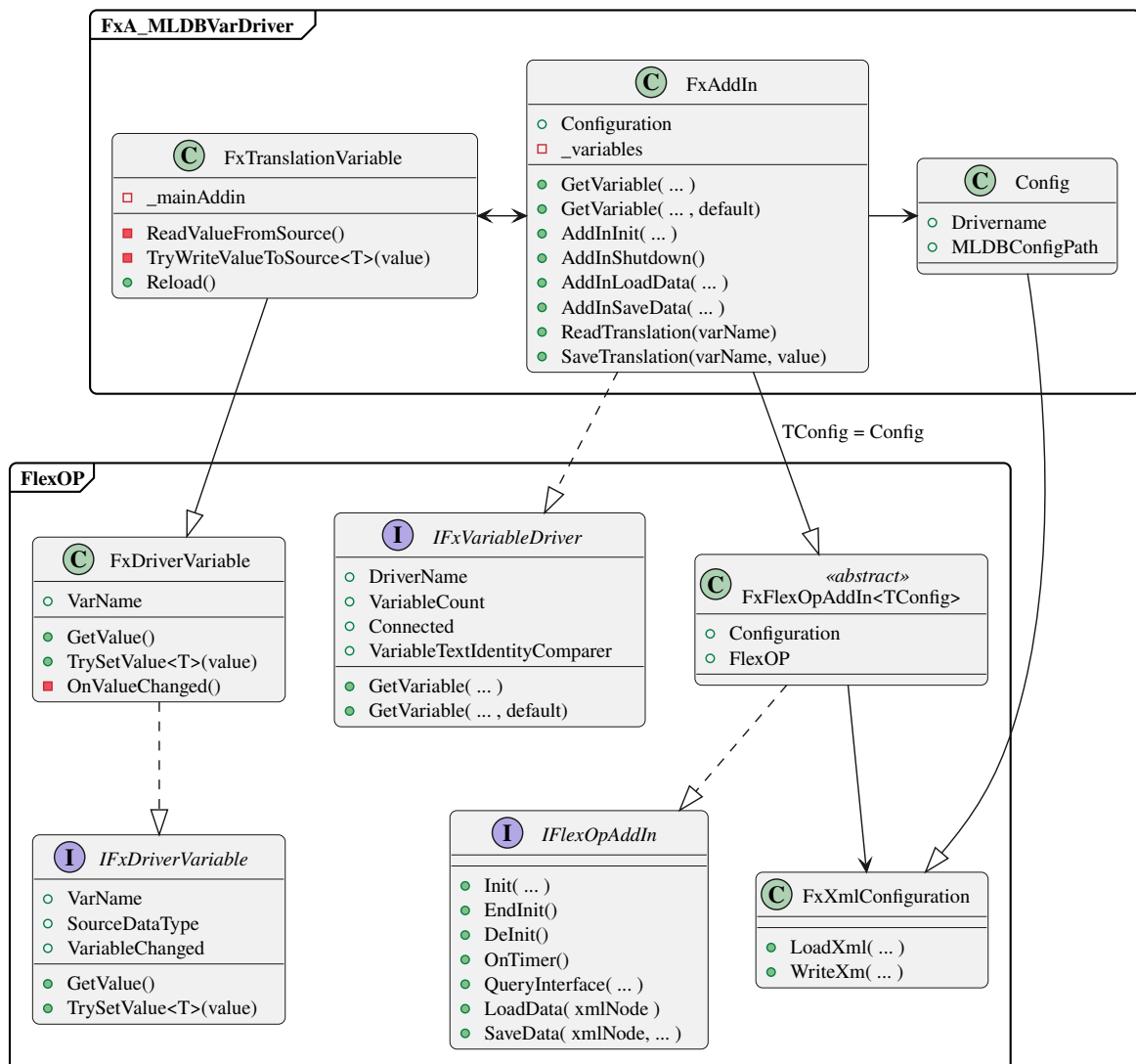


Abb. 3.1: Klassendiagramm der Einbindung in FlexOP

Im „IFxFlexOpAddIn“-Interface, das von der AddIn-Klasse implementiert wird, werden die nötigen Member für die Interaktion zwischen den AddIns und dem AddInProvider definiert. Obwohl Datentreiber aufgrund der Vererbung als vollwertige AddIns verwendet werden können, werden diese im Gegensatz zu „normalen“ AddIns nicht vom AddInProvider verwaltet, da das bereits vom VariableProvider übernommen wird.

3.2.2 Eintragung in die Projektdatenbank

Um programmierte Datentreiber in FlexOP verfügbar zu machen, muss in den Projektdatenbanken ein Verweis auf den Treiber angelegt werden. In diesem Eintrag befinden sich alle Informationen die FlexOP beim Start zum Instanzieren des Datentreibers benötigt. Die folgende Tabelle stellt die Default-Verweise vom OPCUA-Treiber und dem neuen MLDB-Treiber gegenüber.

strName	OPC	MLDB
strProgId	FxA_OPCUAClient.FxAddIn	FxA_MLDBVarDriver.FxAddIn
strAssembly	%FLX_FLXDIR%/F-xA_OPCUAClient.dll	%FLX_FLXDIR%/F-xA_MLDBVarDriver.dll
strParam	%FLX_PRJDIR%/FxA_OPCUAD-AClient.config.xml	%FLX_PRJDIR%/FxA_MLDBVarDriver.Config.xml

Abb. 3.2: Einbindung von Datentreibern in der FxBase-Datenbank

Die Zelle strAssembly gibt den Pfad zu einer dll-Datei an, in der der kompilierte Datentreiber gespeichert ist. Aus dieser Datei wird der Treiber beim Start von FlexOP dynamisch geladen. Hierzu ist zudem noch das Feld strProgId nötig. Dieses Feld gibt den Datentyp des Treibers im Format „Namespace.Klassenname“ an. Zur Instanziierung der Treiber ist noch eine XML-Konfigurationsdatei nötig. Über das strParam-Feld wird der nötige Dateipfad angegeben.

In den Pfaden zur Assembly und Konfiguration können spezielle Platzhalter wie beispielsweise „%FLX_PRJDIR%“ verwendet werden. Diese werden dann von FlexOP aufgelöst. Dadurch werden angegebene Pfade kürzer und aussagekräftiger, zudem ist es möglich das Projekt auf verschiedenen Geräten zu verwenden, unabhängig vom Speicherort im Dateisystem.

Der Name des Treibers kann dann im Rest der Datenbank verwendet werden, um anzuzeigen, über welchen Datentreiber eine Variable abzurufen ist. Dies geschieht über das Format „[Treibername:Variablenname]“, also greift „[MLDB:Test]“ über den MLDB-Treiber auf die Übersetzung der Test-Variable in der Datenbank zu.

3.3 XML-Konfiguration

Bevor in den folgenden Unterkapiteln aufgezeigt wird, welche Konfigurationsparameter für den Datentreiber festgelegt wurden und weshalb die Konfiguration in zwei Dateien unterteilt wurde, wird hier kurz die grobe Funktionsweise beschrieben.

Wie bereits erwähnt, erbt jedes AddIn in FlexOP von der abstrakten Klasse „FxFlexOpAddIn“. In dem Diagramm aus Abbildung 3.1 ist zudem zu sehen, dass dabei der genaue Datentyp des Generikums „TConfig“ festgelegt wird. So kann ein Großteil der Konfigurationshandhabung, wie beispielsweise das Laden aus einer XML-Datei in der Basisklasse übernommen werden. Nach Konvention von fpt befindet sich die Konfigurationsklasse des neuen Treibers im selben Namespace und wird schlicht „Config“ genannt. Die folgenden Erläuterungen werden anhand der finalen Implementation dieser Klasse getätigt.

Mithilfe des „FxConfigValueNode“-Attributs werden Felder einer Tochterklasse der FxXml-Configuration als Konfigurationsparameter markiert. (Sh. Abb. 3.3) Über das Attribut erhalten diese Felder zudem eine kurze Beschreibung, die als Kommentar die XML-Datei geschrieben

wird, um Anlagenprogrammierer bei der Konfiguration zu unterstützen.

```

1 public class Config : FxXmlConfiguration {
2     [FxConfigValueNode("Name des Treibers , zur eindeutigen
      Identifizierung")]
3     public string Drivename = "";
4     [FxConfigValueNode("Pfad zur XML-Datei , die die
      FxMultilangDBConfig fuer den Treiber enthaelt")]
5     public string MLDBConfigPath = "";
6 }

```

Abb. 3.3: Ausschnitt aus der Klasse FxA_MLDBVarDriver.Config

Wenn beim Ladeversuch keine Konfigurationsdatei am angegebenen Pfad existiert, wird diese automatisch erstellt. Zudem kann über einen Parameter im Methodenaufruf noch eingestellt werden, ob fehlende Parameter in der XML-Datei ergänzt werden sollen. In diesen beiden Fällen wird auch der Initialwert des Parameters im Code, hier jeweils leere Strings, in die Konfiguration eingetragen. Bei einer Neuerstellung sieht die XML-Datei also wie folgt aus.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <Config>
3     <!--Name des Treibers , zur eindeutigen Identifizierung-->
4     <Drivename></Drivename>
5     <!--Pfad zur XML-Datei , die die FxMultilangDBConfig fuer
      den Treiber enthaelt-->
6     <MLDBConfigPath></MLDBConfigPath>
7 </Config>

```

Abb. 3.4: Ausschnitt aus der Default-Konfiguration der Datentreibers

3.3.1 Parameter

Die definierten Konfigurationsparameter lassen sich grob in drei Gruppen unterteilen. Der Name des Treibers, die Parameter zur Verbindung und Arbeit mit der Datenbank und Einstellungen zur Handhabung der zu verwaltenden Übersetzungen.

Der Treibername befindet sich in der als einziger Parameter aus der Tabelle 3.5 in der eigentlichen Konfigurationsdatei des Treibers, die in Abbildung 3.4 dargestellt ist. Der Name dient zur Identifikation des Treibers, beispielsweise in Fehlermeldungen. Die restlichen Parameter befinden sich in einer zweiten Datei, deren Pfad konfigurierbar ist.

Zur Verbindung mit Datenbanken ist zunächst ein sogenannter Connectionstring nötig. Dieser enthält unter anderem Name und Adresse/Dateipfad der Datenbank, sowie Informationen

zum Datenbankprovider und Zugangsdaten für passwortgeschützte Datenbanken. Zudem benötigt der Treiber noch den Tabellen- und die Spaltennamen. Die Datentypen der Spalten sind nicht konfigurierbar, sondern immer Integer beziehungsweise Strings oder Booleans.

Allgemeine Einstellungen	
Drivername	Name des Treibers, zur eindeutigen Identifizierung
Datenbankkonfiguration	
ConnectionString	ConnectionString der Datenbank
TableName	Tabelle für Übersetzungen
IdColumn	Spalte zum Speichern der ID
VarNameColumn	Spalte zum Speichern des Variablennamens
NativeTranslationColumn	Spaltenname für deutsche Texte
TranslationColumnPrefix	Präfix des Spaltennamens, für die restlichen Texte
ValidColumnFormat	Spalte zum Speichern, ob die Übersetzung korrekt/aktuell ist
Handhabung von Übersetzungen	
DeprecatedMode	Modus für Handhabung von veralteten/falschen Übersetzungen
MarkFormat	Format zum 'markieren' veralteter Übersetzungen
ErrorMode	Modus für Handhabung von fehlenden Übersetzungen
FallbackLang	Index der bevorzugten Ersatzsprache
ErrorMessage	Alternativtext zum Anzeigen bei fehlenden Übersetzungen

Abb. 3.5: Konfigurationsparameter des MLDB-Datentreibers

Die Handhabung der Übersetzungen kann über verschiedene Modi gesteuert werden. Für die beiden Modi stehen verschiedene Ausprägungen zur Verfügung, die im Code als Enumerationen implementiert sind. Beim Laden der Konfiguration werden die Strings aus der Datei nach Möglichkeit zum passenden Datentyp umgewandelt. Eingegebene Zahlen werden dabei als Index in die Enumeration interpretiert, Texte werden über den Namen der Ausprägung zugeordnet. Wenn die Umwandlung fehlschlägt, wird der Defaultwert verwendet und ein Fehler ausgegeben. Für einige Modi sind zusätzliche Konfigurationen notwendig. Dazu gehört ein Format, das auf „veraltete“ (Sh. Kap. 3.5) Übersetzungen angewendet wird. Das Format sollte den Substring „___“ enthalten, der mit dem veralteten Text ersetzt wird. Im Fall, dass die drei Unterstriche fehlen wird nur eine Warnung ins Log geschrieben, da der Treiber weiter funktioniert, es sich aber wahrscheinlich um einen Konfigurationsfehler handelt. Fehlende Übersetzungen können mit einer konfigurierbaren Fehlermeldung oder mit einer Ersatzsprache ersetzt werden. Wenn der Modus zum Ausweichen auf eine andere Übersetzung aktiv ist, wird bevorzugt die Ersatzsprache aus der Konfiguration verwendet. Invaliden Eingaben für den Sprachenindex werden mit „-1“ ersetzt. Eine Konfiguration mit Ersatzindex „-1“ ist valide, da in diesem Sonderfall automatisch eine nicht veraltete Übersetzung als Ersatz ausgewählt wird.

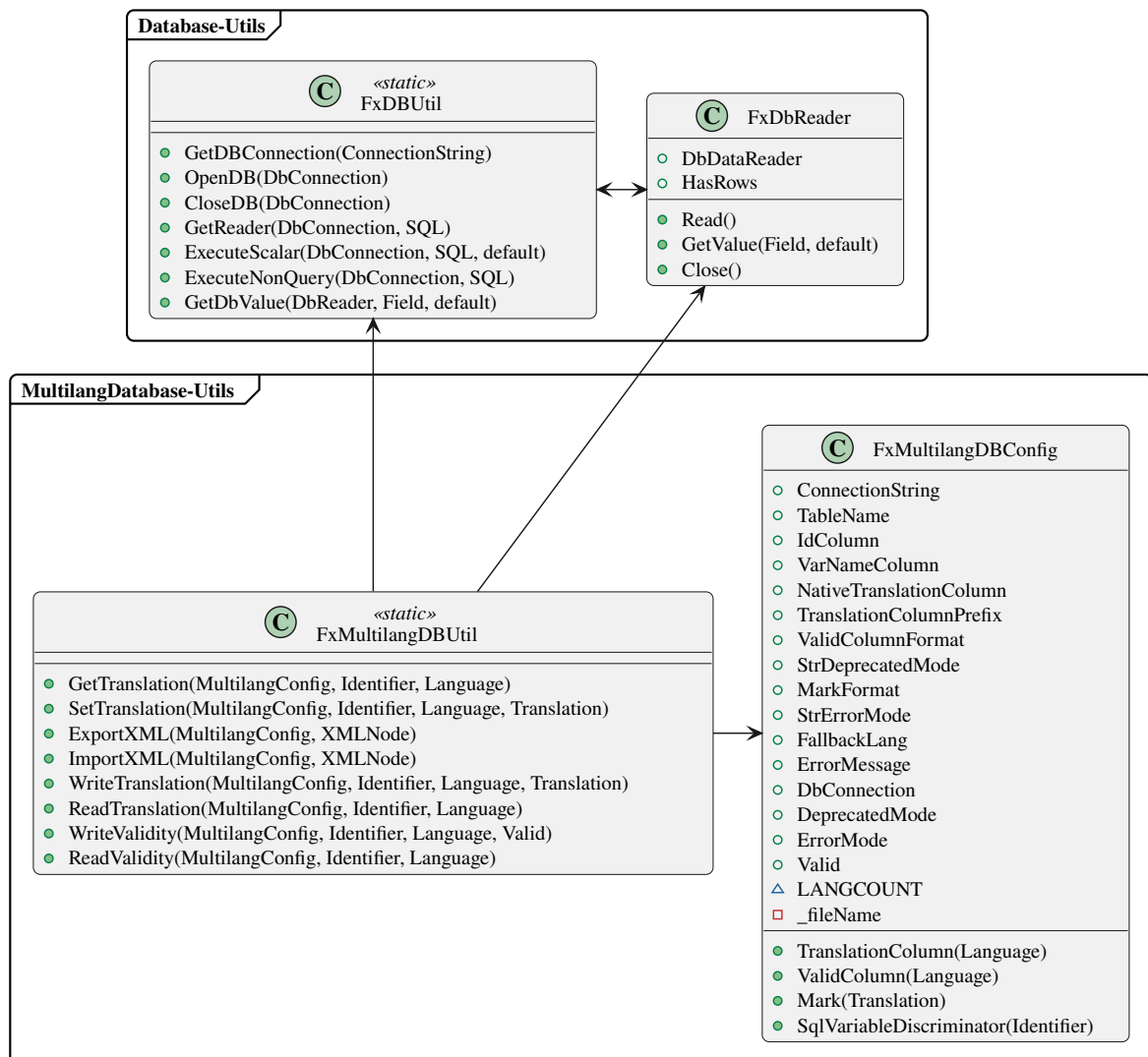


Abb. 3.6: Klassendiagramm der Datenbank-Utilities

3.3.2 Aufteilung in mehrere Dateien

Wie aus der Problemstellung zu erkennen ist, sollen die beiden AddIns für FlexOP „FxA_Hand“ und „FxA_ProgManager“ auch mehrsprachige Tabellen unterstützen. In Kapitel 3.3.1 wurden die dafür nötigen Konfigurationen vorgestellt. Wenn der Treiber diese Parameter in seiner eigenen Konfigurationsklasse definiert hätte, müssten dieselben Parameter auch in die Konfigurationen der beiden AddIns und die aller zukünftigen Komponenten eingefügt werden. Dank der Aufteilung in mehrere Dokumente reicht eine Erweiterung um den Pfad zur zweiten Datei. Das erleichtert die Ergänzung des Features in neuen und den vorhandenen AddIns. Zudem können so die Methoden zur Arbeit mit mehrsprachigen Tabellen explizit auf die Konfiguration angepasst werden. Für neue AddIns wird die Erweiterung um mehrere Sprachen damit ähnlich zum „Plug-and-Play“. In den vorhandenen AddIns sind einige weitere Änderungen nötig, da bisher

das Schreiben in der Datenbank von den AddIns selbst übernommen wird, und zudem in den Tabellen der AddIns noch andere Informationen, als die mehrsprachigen Texte, verwaltet werden.

3.4 Datenbankformat

Beim Starten des Treibers wird überprüft, ob die angegebene Tabelle die nötigen Spalten mit den korrekten Datentypen enthält. Im Fall, dass die Tabelle nicht existiert, kann einfach eine kompatible Tabelle erstellt werden. Falls die Tabelle jedoch bereits existiert und Spalten fehlen beziehungsweise falsche Datentypen nutzen, schreibt der Treiber eine Fehlermeldung in die Log-Datei und gibt bei Anfragen Fehlermeldungen zurück. Bei korrekter Konfiguration beginnt der Treiber angefragte Variablen in der Datenbank anzulegen und zu verwalten.

ID	VarName	Native	Lang0Ok	LangL1	Lang1Ok	...
1	Msg1	Hallo	True	Hello	True	...
2	Msg2	Tschüss	True	Goodbye	False	...

Abb. 3.7: Beispielinhalt der Datenbank

Abbildung 3.7 zeigt eine exemplarische Tabelle, mithilfe der im Folgenden das Format kurz erklärt wird.

Die Spaltennamen für ID und Variablenname werden direkt aus der Konfigurationsdatei übernommen. Beide Spalten dienen zur Identifikation der Variablen. Der Datentreiber identifiziert die korrekte Zeile im Normalfall über den Variablennamen und generiert für jede Zeile eine eindeutige ID. Die ID wird für ein Firmeninternes Tool der fpt benötigt. Der sogenannte Language-Manager hilft beim Übersetzen der Datenbanken und soll daher auch mit der Tabelle des Treibers kompatibel sein.

Die Spalte mit den deutschen Texten erhält ihren Namen auch direkt aus der Konfiguration. Dies beruht darauf, dass viele andere Tabellen in den Projektdatenbanken spezielle Namen für die deutsche Spalte haben und der Language-Manager dieses Feature unterstützt. Für den Fall, dass der Treiber einmal auf eine dieser Tabellen angewendet werden muss, muss auch der Datentreiber Sondernamen für die Nativspalte unterstützen. Für die anderen Spalten, die Übersetzungen enthalten kann in der Konfiguration nur ein Präfix festgelegt werden. Diese Limitation ist ebenfalls auf den Language-Manager zurückzuführen, den dieser Benötigt für jede Sprachspalte, die nicht die deutsche, ist ein Suffix im Format: „L{Sprachindex}“. Um die Kompatibilität mit dem Übersetzungstool zu garantieren, hängt der Datentreiber das nötige Suffix an den Spaltennamen an, wie in Abbildung 3.7 zu sehen ist. Da Deutsch immer den Sprachindex null hat, ist der Spaltenname für die englischen Übersetzungen hier „LangL1“, mit „Lang“ als konfiguriertem Präfix.

Die „ValidColumn“ für jede Sprache, hier „Lang00k“ bei Deutsch, verwendet der Datentreiber zum Markieren veralteter Übersetzungen. In der Konfigurationsdatei wird für diese Spalten ein Format angegeben, bei dem der Platzhalter „__“ mit dem Index der Sprache ersetzt wird. Der Language-Manager verwendet diese Spalten nicht, wenn über den Language-Manager also eine Übersetzung korrigiert wird bleibt sie weiterhin als veraltet markiert. Diesen Fehler im Treiber zu korrigieren ist nicht möglich, da das Übersetzungstool im Normalfall nur an der Datenbank arbeitet, während FlexOP geschlossen ist und demnach der Datentreiber nicht aktiv ist. Nur durch Integration der Valid-Spalte im Language-Manager kann dieser Ausnahmefall eliminiert werden. Änderungen am Language-Manager sind jedoch nicht teil dieses Projektes.

3.5 Handhabung der Texte

Das Verhalten des Treibers wird über die beiden Konfigurationsparameter „ErrorMode“ und „DeprecatedMode“ gesteuert. Dabei stehen, wie bereits in Kapitel 3.3.1 erwähnt wurde, jeweils verschiedene Ausprägungen der Enums zur Verfügung. Im Folgenden wird genau dargestellt, wie die in Abbildung 3.8 zu sehenden Modi und zugehörige Konfigurationsparameter die Handhabung der Übersetzungen beeinflussen.

ErrorMode	
└ melden	ErrorMessage
└ ersetzen	FallbackLang
DeprecatedMode	
└ behalten	
└ ueberschreiben	
└ loeschen	
└ markieren	MarkFormat

Abb. 3.8: Konfigurationsparameter zur Handhabung von Texten

3.5.1 ErrorMode

Für den ErrorMode regelt das Verhalten beim Zugriff auf fehlende Übersetzungen. Wenn der Parameter auf „melden“ gesetzt ist, wird anstelle der leeren Tabellenzelle eine Fehlermeldung zurückgegeben. Der Meldungstext erscheint dann in dem Textfeld, in dem eigentlich die Übersetzung angezeigt wird. So bemerkt der Anwender, dass der Text fehlt und kann durch Eintragen eine Übersetzung festlegen. Die Fehlermeldung kann über den Parameter „ErrorMessage“ konfiguriert werden, mithilfe einer leeren Fehlermeldung kann also das Textfeld auch leer gelassen werden.

Die andere mögliche Ausprägung des ErrorMode ist „ersetzen“. In diesem Fall wird, anstatt einer Meldung, der Text aus einer anderen Sprache angezeigt. Mithilfe des Parameters

„FallbackLang“ kann konfiguriert werden, welche Sprache bevorzugt zum Ersetzen verwendet wird. Wenn die Übersetzung der angegebenen Ersatzsprache auch leer ist, wird vom Datentreiber automatisch eine nicht veraltete Sprache ausgewählt. Falls alle Texte veraltet sind, was unter gewöhnlichen Umständen nicht vorkommt, sondern nur, wenn manuell Änderungen in der Tabelle vorgenommen werden, wird die deutsche Übersetzung zurückgegeben.

Im Sonderfall, dass die Ersatzsprache mit dem Index „-1“ konfiguriert ist, wird nicht zuerst geprüft, ob eine Übersetzung in der Ersatzsprache existiert, sondern direkt ein nicht veralteter Text verwendet.

3.5.2 DeprecatedMode

Über den DeprecatedMode wird konfiguriert, wie der Datentreiber mit veralteten Übersetzungen umgeht. Wenn in FlexOP ein neuer Text für eine Variable des Treibers eingegeben wird, wird der neue Text in der korrekten Tabellenzelle gespeichert und als nicht veraltet gekennzeichnet. Gleichzeitig werden alle anderen gespeicherten Übersetzungen der Variable als veraltet markiert. Das ist nötig, für den Fall, dass sich die Bedeutung des neuen Textes von der Bedeutung der alten Übersetzungen unterscheidet. In der Tabelle 3.7 ist zu sehen, dass der englische Text als veraltet gekennzeichnet wurde, nachdem die deutsche Übersetzung zu „Tschüss“ geändert wurde. Hier unterscheiden sich zwar nicht die Bedeutungen, vorsichtshalber werden diese Markierungen dennoch vorgenommen.

Die erste Möglichkeit für den Modus ist „behalten“. In dieser Konfiguration werden veraltete Texte genau gleich wie nicht veraltete Texte angezeigt. So ist die angezeigte Sprache immer korrekt, aber die Bedeutung kann verfälscht sein. Im Gegensatz dazu werden im „ueberschreiben“-Modus alle Übersetzungen mit der neu eingetragenen überschrieben. So wird garantiert immer die neueste Version angezeigt, allerdings in der falschen Sprache. Eine Alternative zum Überschreiben bietet die Ausprägung „loeschen“. Hier werden veraltete Übersetzungen aus der Datenbank gelöscht und mithilfe des ErrorMode kann eine Fehlermeldung anstatt veralteten Texten angezeigt werden.

Der letzte Modus ist „markieren“. Veraltete Übersetzungen werden mithilfe des Konfigurationsparameters „MarkFormat“ für den Anwender als veraltet angezeigt. Beispielsweise würde mit dem Format „[___]“ „Goodbye“ aus der Tabelle 3.7 als „[Goodbye]“ in der englischen Bedieneroberfläche angezeigt werden. So werden Übersetzungen immer in der richtigen Sprache, und meist auch mit der richtigen Bedeutung angezeigt, für Ausnahmefälle ist der Text aber als nicht garantiert vertrauenswürdig erkennbar. Wenn das Format nicht den Platzhalter „___“ enthält können über den „markieren“-Modus auch Meldungen anstatt veralteten Variablen angezeigt werden, ohne die Einträge in der Datenbank zu löschen.

4 Umsetzung

Nach der Darstellung des Implementationsentwurfs wird nun kurz aufgezeigt welche Hindernisse bei der Implementation festgestellt wurden und zudem, wie die jeweiligen Probleme bewältigt wurden.

4.1 Ex- und Importieren der AddIn-Daten

AddIns in FlexOP sind in der Lage Datensätze, wie Konfigurationen oder Speicherdaten, in eine XML-Datei zu exportieren und sie auch wieder einzulesen. Dieses Feature wird dazu verwendet, wenn zwischen verschiedenen Icon-Programmen in FlexOP gewechselt wird, es dient aber auch dazu die genauen Einstellungen in FlexOP von einem System auf ein anderes zu kopieren.

Datentreiber in FlexOP erweitern, wie in Abbildung 3.1 zu sehen ist, die AddIn-Klasse. Somit sind auch die Treiber in der Lage, Daten zu importieren beziehungsweise zu exportieren. Bei der Implementation wurde jedoch vermutlich nicht bedacht, dass Datentreiber nicht in derselben Liste, wie AddIns gespeichert werden, und somit nur AddIns ihre Daten ex- und importiert haben. Dieser Umstand ist bis zu diesem Projekt nicht aufgefallen, da bisher alle Datentreiber das Speicherfeature nicht genutzt haben.

Da je nach Icon-Programm jedoch andere Texte in FlexOP angezeigt werden sollen, muss der neue Datentreiber beim Programmwechsel die alten Übersetzungen aus der Datenbank in der XML-Datei speichern und die neuen Texte aus der Speicherdatei in die Datenbank schreiben.

Zum Beheben des Bugs muss neben der Liste mit den AddIns beim Lesen und Schreiben der Speicherdatei auch über die Liste der Datentreiber iteriert werden. Zur Zuordnung welcher Teil der Speicherdatei zu welchem AddIn gehört, wurde der Abschnitt der XML-Datei bisher mit den Namen des AddIns, aus der Projektdatenbank, gekennzeichnet. Für Datentreiber wird der Name unter „IFxVariableDriver.DriverName“ verwendet. Das erfordert, dass alle Datentreiber einen eindeutigen Namen erhalten. Das ist immer der Fall, da der VariableProvider den Namen auch zu Identifikation der Treiber verwendet.

4.2 Kompatibilität mit verschiedenen Dotnet-Versionen

FlexOP kann für verschiedene Versionen der Plattform „.NET“ kompiliert werden. Seit der Version „.NET6.0“ ist die Klasse „OleDbConnection“ nur noch mit dem Betriebssystem Windows kompatibel. Um unerwartete Fehler beim Verwenden von FlexOP auf anderen Betriebssystemen zu verhindern, muss beim Kompilieren für Dotnet-Versionen sechs oder neuere, vom Präprozessor eine Abfrage des Betriebssystems in den Code eingefügt werden.

```
1 ( ... )
2 #if NET6_0_OR_GREATER
3 if ( ! OperatingSystem . IsWindows ( ) )
4 {
5     FxTrace . LogErr ( " Seit _NET6_ ist _OleDbConnection _nur _auf _
        Windows _supportet " ) ;
6     return null ;
7 }
8 #endif
9 ( ... )
```

Abb. 4.1: Betriebssystemprüfung für Versionen NET6 und neuere

Mit einem „#“ am Beginn der Codezeile werden in C# Präprozessorbefehle gekennzeichnet. Hier wird der Code zwischen „#if NET6_0_OR_GREATER“ und „#endif“ also nur eingefügt, wenn FlexOP für Dotnet-Versionen kompiliert wird, in denen die OleDbConnection nur noch auf Windows unterstützt wird. (Microsoft Corporation 2023)

Der eingefügte Code generiert eine Fehlermeldung, wenn versucht wird die OleDbConnection zu verwenden, obwohl sie nicht verfügbar ist. Als Alternativen stehen hier die Klassen „SqlConnection“ und „OdbcConnection“ zur Verfügung. Beide sind auch in neuen Versionen auf allen Betriebssystemen unterstützt.

4.3 Identifikation über ID anstelle vom Namen

Der Datentreiber identifiziert Variablen, und somit Einträge in der Datenbank über den Variablennamen. Die AddIns für Teachpositionen und Icons hingegen verwenden IDs. Auch zukünftige Anwendungen sollen Einträge in mehrsprachigen Tabellen über die ID bestimmen können.

Hierzu wird die Konfigurationsklasse um eine Methode erweitert, mit der die „WHERE“-Clause des SQL-Kommandos generiert werden kann. Die Methode erhält als Input ein Objekt von beliebigem Datentyp. Ausgehend davon, ob es sich bei dem Input um einen Integer oder String handelt, wird im Rückgabewert die „IdColumn“ oder die „VarNameColumn“ aus der Konfiguration verwendet. Wenn der Input weder eine Zahl noch ein Text ist, wird eine Exception geworfen, da ein solcher Fehler nur entstehen kann, wenn bei der Programmierung eines neuen AddIns ein fataler Fehler unterlaufen ist. Außerdem ist das Logfile für Fehler in der Konfiguration von FlexOP vorgesehen.

In Anwendungsfällen, in denen Einträge über die ID anstelle des Namens ausgewählt werden, ist es möglich, dass keine „VarNameColumn“ angegeben werden kann, wie im Fall der AddIns „FxA_ProgManager“ und „FxA_Hand“. In den Konfigurationsdateien solcher AddIns kann der Parameter „VarNameColumn“ leer gelassen werden. Bei der Überprüfung des Tabellenformats wird dann die Spalte für Variablennamen nicht gesucht und bei einer Neuerstellung der Tabelle

auch nicht angelegt.

Dadurch, dass die Spalte für Variablennamen optional ist, entsteht eine neue Fehlerquelle. Eine Anwendung, die Variablen über den Namen identifiziert, kann mit leerem „VarNameColumn“ konfiguriert werden. Der Konfigurationsfehler wird erst beim Zugriff auf eine Variable erkannt. In der Methode zum Erstellen der „WHERE“-Clause wird bei der Verwendung eines Strings als Input geprüft, ob eine Namensspalte konfiguriert ist und dementsprechend eine Meldung ins Log geschrieben. Im Fehlerfall wird als Bedingung für das „WHERE“-Statement „False“ zurückgegeben. Andernfalls wäre der generierte SQL-Befehl, in der Form „... WHERE = {VarName}“, nicht syntaktisch korrekt und würde zu Fehlern führen.

Eine logische Erweiterung zur optionalen „VarNameColumn“ wäre, die „IdColumn“ auch optional zu machen, mitunter weil eine der beiden Spalten als Primärschlüssel für die Tabelle ausreicht. Da der Language-Manager jedoch eine ID in den verwendeten Tabellen benötigt und alle bisherigen Tabellen eine Spalte für IDs besitzen, bleibt die Spalte für IDs in der Konfiguration verpflichtend. Dies hat zudem den Vorteil, dass immer die ID als Primärschlüssel beim Erstellen von Tabellen verwendet werden kann und somit der Code vereinfacht wird.

4.4 Konflikte zwischen Zugriffsmethoden auf die Datenbank

Das AddIn „FxA_ProgManager“ verwendet Member des „dao“-Namespace (Microsoft Corporation 2021) zum Bearbeiten von Tabelleninhalten. Der Namespace ist zum Entwickeln von Erweiterungen für die Applikationen „Access 2013“ und „Office 2013“ vorgesehen. Dabei wird ein sogenanntes „Recordset“ verwendet. Das Set repräsentiert den Inhalt einer Tabelle und erlaubt Zugriffe auf Subelemente ähnlich wie eine Liste von Objekten.

Zum Zeitpunkt dieser Projektarbeit bestehen bereits Pläne das AddIn umzuschreiben, und stattdessen die neuen Methoden aus dem Namespace „Fx.FlexOPFramework.Util.Databases“ zu verwenden. Dieselben Methoden werden auch im MLDB-Treiber verwendet. Das AddIn „FxA_Hand“, bei dem die Mehrsprachigkeit ohne Probleme integriert werden konnte, verwendet eine eigene Implementation der Methoden, basierend auf denselben Grundlagen. Hier wird ausgehend von den Informationen zur Datenbank eine „DbConnection“ erstellt. Mithilfe von der Datenbankverbindung können Befehle in der Sprache „Structured Query Language“ (SQL) an der Datenbank ausgeführt werden.

Beim Implementieren der Mehrsprachigkeit in den ProgManager wird also über zwei verschiedene Methoden gleichzeitig auf dieselbe Tabelle zugegriffen. In einem solchen Szenario muss sichergestellt werden, dass die Tabelle von keinem der beiden Namespaces gesperrt wird, da sonst keine Änderungen oder sonstige Zugriffe über den anderen Namespace möglich sind. Dennoch entsteht dabei der Fehler, dass eine der beiden Zugriffsarten scheitert und Exceptions wirft. Die angezeigten Fehlermeldungen sind nichtssagend und auch im Internet ist keine Infor-

mationen zu Konflikten zwischen Datenbankzugriffen über die beiden Namespaces zu finden. Wie Änderungen im Code beeinflussen, welche der beiden Methodiken fehlschlägt, ist zudem scheinbar zufällig.

Nach Beratung mit Simon Würzer, der ebenfalls keine Lösung für das Problem wusste, wurde daraufhin beschlossen mit der Implementation der Mehrsprachigkeit für die Icon-Programme zu warten, bis der ProgManager umgeschrieben wurde. Das zugehörige Issue auf Github existiert jedoch bereits seit Ende 2021 und wird zurzeit von Amrita Hedge. (Sh. github.com/FPTSystems/Fx.FlexOP/issues/755)

5 Qualitätskontrolle

Um Sicherzustellen, dass der Datentreiber und die anderen relevanten Methoden planmäßig funktionieren, muss die Implementation in verschiedenen Szenarien getestet werden. Typischerweise werden solche Tests automatisch durchgeführt. Zum Initialisieren von AddIns ist jedoch eine Referenz auf eine FlexOP-Instanz nötig. Für Testzwecke ist es möglich, eine „simulierte“ Version von FlexOP, die alle nötigen Interfaces implementiert, einzurichten.

Der Datentreiber verwendet jedoch ein paar Methoden, die nicht in der vorgefertigten Testversion von FlexOP implementiert sind und nachträglich ergänzt werden müssten. Daher werden die formulierten Tests für dieses Projekt manuell in einem Basisprojekt von FlexOP durchgeführt.

5.1 Hilfreiche Features

Um die Tests möglichst effizient durchführen zu können, wurde in den Datentreiber ein Mechanismus implementiert, der Automatisch die Konfigurationsdatei und die Texte in der Datenbank neu einließt. Diese Features sind zudem sehr hilfreich bei der Konfiguration des Treibers, daher werden sie nach der Fertigstellung des Codes nicht deaktiviert.

5.1.1 Neuladen der Konfiguration

AddIns verfügen über eine Methode, die Zyklisch von FlexOP aufgerufen wird. In dieser Methode prüft der Datentreiber, den letzten Speicherzeitpunkt der Konfigurationsdatei mit den Einstellungen zur Handhabung der Übersetzungen (Sh. Abb. 3.5). Ist der Speicherzeitpunkt später als der gespeicherte Zeitstempel vom letzten Laden der Konfiguration, wird die Konfiguration neu eingelesen.

Hier kann es dazu kommen, dass Konfigurationsfehler in der neuen Datei sind. Deshalb wird die alte Konfiguration nicht sofort mit der neuen ersetzt. Zuerst wird geprüft, ob die neu eingelesene Version Fehler enthält. Wenn die neue Version fehlerfrei ist, wird sie übernommen, andernfalls wird eine Fehlermeldung in die Log-Datei geschrieben.

Wie in Kapitel 4.3 erwähnt wurde, erkennt die Konfiguration nicht den Fehler, wenn in einer Anwendung, die Variablennamen verwendet, kein Spaltenname für die „VarNameColumn“ konfiguriert ist. Dazu muss im Treiber gesondert geprüft werden, dass ein Name für die Spalte festgelegt ist, um sicherzugehen, dass die Konfiguration im Datentreiber funktioniert.

Durch das Neuladen der Datei können mehrere Konfigurationen getestet werden, ohne dass FlexOP neu gestartet werden muss, was oftmals über eine halbe Minute dauert.

5.1.2 Neuladen der Datenbankeinträge

Der Datentreiber speichert neue Texte in der aktuellen Sprache von FlexOP. Beim Ändern der Sprache ist in FlexOP jedoch ein Neustart nötig. Um also Texte für Deutsch und die fünf Fremdsprachen über den Datentreiber in die Datenbank einzutragen, sind sechs Neustarts von FlexOP nötig. Alternativ können die Werte von Hand in die Datenbank eingetragen werden, dann ist nur noch ein Neustart nötig, dass die neuen Texte in FlexOP angezeigt werden.

Die „FxDriverVariable“ kann über die private Methode „OnValueChanged()“ (Sh. Abb. 3.1) Signalisieren, dass sich der Wert der Variable geändert hat. Diese Methode wird über die Methode „Reload()“ in der Übersetzungsvariable öffentlich gemacht, sodass der Treiber auch signalisieren kann, dass sich der Wert geändert hat.

Ähnlich wie in Kapitel 5.1.1 überprüft der MLDB-Treiber zyklisch, ob eine neue Version der Datenbank gespeichert wurde. Wenn das der Fall ist, wird in allen verwendeten Variablen des Treibers die Methode zum neuen Laden aufgelöst. Dazu müssen alle Variablen, die vom Treiber erstellt wurden in einer Liste gespeichert im Treiber zugänglich sein. Mithilfe der Liste werden zudem nur Einträge aus der Datenbank, die auch verwendet werden, neu eingelesen.

5.2 Tests

Um sicherzugehen, dass alle nötigen Tests durchgeführt werden, wird eine Liste der zu testenden Aspekte erstellt. Die Liste lässt sich grob in vier Teile unterteilen, die XML-Konfiguration, die Datenbank, das Verhalten des Treibers und das Importieren und Exportieren von Daten.

5.2.1 XML-Konfiguration

Der wichtigste Test bei der Konfiguration, ist sicherzustellen, dass ungültige Konfigurationen auf keinen Fall den Start von FlexOP verhindern, beziehungsweise die Funktion anderer Teile der Software einschränkt. Andernfalls ist es möglich, dass die Log-Datei nicht korrekt befüllt wird, beziehungsweise viele Meldungen von Folgefehlern enthält und somit unlesbar wird. Zum Testen muss der Treiber mit verschiedenen ungültigen Konfigurationen gestartet werden, hierzu kann die Validitätsprüfung aus Kapitel 5.1.1 temporär deaktiviert werden, um Zeit zu sparen. Kategorien bei Konfigurationsfehlern, die hierzu getestet werden, sind unter anderem, ein fehlender Treibername, ein ungültiger Dateipfad zur zweiten Konfigurationsdatei, sowie diverse Fehler in der „FxMultilangDBConfig“. Zudem wird getestet, dass fehlende Konfigurationsdateien automatisch an den angegebenen Pfaden generiert werden.

Das Einlesen simpler Datentypen aus Konfigurationsdateien wird in der bereits vorhandenen und getesteten Basisklasse gehandhabt und muss daher nicht explizit getestet werden. Stattdessen muss sichergestellt werden, dass Zahlen und Texte korrekt zu den Enums „DeprecatedMode“

und „ErrorMode“ geparkt werden und Platzhalter wie „%FLX_PRJDIR%“ in Dateipfaden korrekt mithilfe von FlexOP aufgelöst werden.

5.2.2 Datenbank

In der Datenbank wird zuerst getestet, ob eine fehlende Tabelle automatisch korrekt erstellt wird. Dazu gehört auch das automatische Anlegen eines Index auf die „VarNameColumn“, um SQL-Anfragen zu beschleunigen. In der erstellten Tabelle wird anschließend getestet, dass die Verbindung des Datentreibers funktioniert, und Daten prinzipiell gelesen und geschrieben werden können. Dabei wird zudem sichergestellt, dass durch Eingeben von bestimmten Texten in FlexOP keine SQL-Injektion möglich ist. Zuletzt werden verschiedene Spalten aus der Tabelle gelöscht, beziehungsweise deren Datentyp geändert, um zu prüfen, dass inkompatible Tabellen vom Treiber frühzeitig erkannt werden und keine unerwarteten Fehler erzeugen.

5.2.3 Verhalten des Treibers

Das Verhalten des Datentreibers wird durch die Konfigurationsparameter „DeprecatedMode“, „ErrorMode“ und die dazugehörigen Hilfsparameter aus Abbildung 3.8. Zum vollständigen Testen müssten also alle möglichen Kombinationen der Konfigurationsparameter getestet werden. Aus Zeitgründen ist das nicht möglich. Stattdessen werden die Modi möglichst unabhängig voneinander getestet.

Zum Testen der verschiedenen Ausprägungen des „DeprecatedMode“ werden verschiedenen Variablen in FlexOP eingerichtet und über die Datenbank mit Übersetzungen befüllt. Anschließend werden, in den verschiedenen Konfigurationen, über FlexOP Änderungen an den Texten vorgenommen. Ausgehend von den resultierenden Änderungen in der Datenbank kann beurteilt werden, ob die Implementation der Modi korrekt gelungen ist.

Im „behalten“-Modus ist zu erwarten, dass die neu eingetragene Übersetzung in die Datenbank übernommen und als aktuell markiert wird. Die anderen Übersetzungen bleiben unverändert, werden jedoch als veraltet markiert. Beim Wechsel in eine andere Sprache für FlexOP dürfen die veralteten Übersetzungen nicht markiert angezeigt werden. Im Modus „markieren“ muss das gleiche Ergebnis in der Tabelle zu sehen sein, mit dem Unterschied, dass beim Anzeigen der Variablen das Markierungsformat auf die Texte angewendet wird. Beim „ueberschreiben“ und „loeschen“ ist zu erwarten, dass beim Eintragen einer neuen Übersetzung die anderen jeweils gelöscht, beziehungsweise überschrieben werden.

Die verschiedenen Konfigurationen des Parameters „ErrorMode“ werden getestet, indem die Spalte mit der aktuell ausgewählten Sprache für einige Einträge leer gelassen wird. Im Modus „melden“ muss die konfigurierte Fehlermeldung anstelle der fehlenden Texte angezeigt werden.

Beim Ersetzen fehlender Texte versucht der Treiber zunächst die konfigurierte Ersatzsprache anzuzeigen. Zum Testen wird der Inhalt der Tabelle wie folgt eingerichtet.

ID	VarName	LangL0	Lang0Ok	LangL1	Lang1Ok	LangL2	Lang2Ok	...
1	TestVar1		False	Lang1	True	Lang2	False	...
2	TestVar2		False		False	Lang2	True	...

Abb. 5.1: Testinhalt der Datenbank

Für den Test wird als aktuelle Sprache „0“ ausgewählt, da diese Spalte in der Testtabelle leer gelassen wurde. Die folgende Tabelle zeigt die Indizes der angezeigten Sprachen, die erwartet werden, wenn das Ersetzen korrekt implementiert wurde.

konfigurierte Ersatzsprache	angezeigte Sprache für „TestVar1“	angezeigte Sprache für „TestVar2“
-1	1	2
0	1	2
1	1	2
2	2	2

Abb. 5.2: Erwartete Testergebnisse

Wie in der Tabelle 5.2 zu sehen ist, wird „TestVar2“ immer in der zweiten Sprache angezeigt, das liegt daran, dass nur für Sprache „2“ eine Übersetzung definiert wird und somit die Ersatzsprache entweder auf eine leere Übersetzung zeigt und somit ignoriert wird, oder die Ersatzsprache hat den Wert „2“. Für „TestVar1“ wird immer Sprache „1“ angezeigt, da es die übersetzte Sprache ist, die der Treiber findet, es sei denn als Ersatzsprache ist die zweite Sprache ausgewählt. In diesem Fall wird die veraltete Übersetzung „Lang2“ angezeigt, da der Treiber angewiesen ist bevorzugt die zweite Sprache zum Ersetzen zu verwenden, und eine Übersetzung für die Sprache existiert.

5.2.4 Import und Export

Beim Exportieren und Importieren der Daten aus dem Treiber, ist essenziell, dass mithilfe der exportierten XML-Datei alle relevanten Informationen aus der Tabelle wiederhergestellt werden. Zum Testen wird die Tabelle also mit verschiedenen Texten befüllt. Anschließend werden die Daten über FlexOP exportiert. Nachdem die Tabelle gelöscht wurde und die exportierten Daten wieder über FlexOP eingelesen wurden, muss der Inhalt der Tabelle identisch zum vorherigen Inhalt sein.

Erlaubte Ausnahmen hierzu sind die Reihenfolge der Einträge und Variablen, bei denen alle Übersetzungen leer sind. Die Reihenfolge ist irrelevant, da Einträge immer über die ID, bezie-

hungsweise den Namen und nicht ihre Position in der Tabelle identifiziert werden. Leere Variablen müssen nicht exportiert und wieder importiert werden, da die Defaultwerte beim Erstellen einer neuen Variable sechs leere Textfelder sind. Somit gehen keine Informationen verloren, wenn die weggelassenen Variablen neu erstellt werden müssen.

6 Reflexion

Nach der Darstellung der Projektarbeit wird im Folgenden der erreichte Stand zusammengefasst und kritisch beurteilt. Zudem werden noch einige mögliche Erweiterungen dargestellt, sowie die Gründe, weshalb sie nicht in das Projekt aufgenommen wurden.

6.1 Zusammenfassung

Die Methoden zur Arbeit mit Datenbanken aus dem AddIn „FxFlexParam2“ wurden von Visual Basic zu C# übersetzt und in FlexOP zugänglich gemacht. Der Methodensatz wurde um weitere Methoden zum Bearbeiten von mehrsprachigen Tabellen erweitert. Zudem wurde eine Klasse für die nötigen Konfigurationen entworfen und implementiert. Mithilfe der neuen Methoden und Klassen wurde ein Datentreiber zur Verwaltung von mehrsprachigen Textvariablen entwickelt. Die Funktionalität wurde zudem in das AddIn „FxA_Hand“ integriert. Das AddIn wurde so erweitert, dass die Namen von Teachpositionen in verschiedenen Sprachen verfügbar sind. Die Integration in das AddIn „FxA_ProgManager“ wurde, aufgrund von Problemen bei der Verbindung mit der Datenbank verschoben, bis der ProgManager umgeschrieben wird und sich mithilfe der neuen Datenbankmethoden mit der Datenbank verbindet. Zuletzt wurde eine Liste mit Tests zusammengestellt, um Sicherzustellen, dass die Implementation dem Entwurf entspricht.

6.2 Limitationen und mögliche Erweiterungen

Wie bereits in der Zusammenfassung erwähnt wurde der ProgManager nicht auf Mehrsprachigkeit erweitert. Stattdessen wird das AddIn erst umgeschrieben, um eine weniger veraltete Methodik zur Arbeit mit Datenbanken zu verwenden. Die neue Methodik, unter anderem über den Namespace „System.Data.SqlClient“, ist jedoch auch nicht aktuell und erhält seit 2019 nur noch Bugfixes. (Harp und Vega 2019)

Die Anzahl der Sprachen, die mithilfe der neuen Methoden verwendbar ist, wird in der Konfigurationsklasse über eine Konstante festgelegt. So können zwar Erweiterungen auf eine höhere Maximalzahl Sprachen einfach durch Anpassen der Konstante im Code vorgenommen werden. Für eine Erweiterung auf potenziell unbegrenzte Sprachen sind größere Änderungen am Code nötig. Diese Implementation wurde gewählt, um der Anforderung nur fünf Fremdsprachen zu Unterstützen nachzukommen, und dennoch die Menge technischer Schulden möglichst minimal zu halten.

Die Tests in diesem Projekt wurden in einer Liste festgehalten, und bei Änderungen im Code manuell durchgeführt. Stattdessen könnten mithilfe einer automatisierten Test-App viel mehr verschiedene Tests durchgeführt werden. Die schnellere Durchführung von Tests spart zudem

Zeit, die wiederum zum Implementieren weiterer Features verwendet werden kann. Das Ausführen der Tests kann zudem in die Build-Pipeline oder via „GitHub Actions“ in die Versionsverwaltung integriert werden. So würden die Tests regelmäßig automatisch durchgeführt.

Der Datentreiber verwaltet nur Texte, die entweder von Anlagenprogrammierern, Übersetzern oder Kunden in verschiedenen Sprachen eingegeben werden. Eine mögliche Erweiterung der Funktionalität wäre automatische Übersetzungen in den Datentreiber zu integrieren. Hierzu könnten Übersetzungsserver, wie die der Firma „DeepL SE“, verwendet werden. Solche Services sind jedoch kostenpflichtig und benötigen eine Verbindung zum Internet. Im Normalfall verfügen Anlagen jedoch über keine Internetverbindung, abgesehen von speziellen VPN-Tunneln für die Fernwartung. Deshalb wurde dieses Feature nicht in den Treiber implementiert.

Standardmäßig ist die Verwendung der Übersetzungsserver an ein Abonnement gebunden. Die zusätzlichen Kosten sind für fpt und Kunden vermutlich kein Problem, da bereits Übersetzungsbüros für das Übersetzen der festen Texte bezahlt werden. Diese Kosten könnten mittels automatischer Übersetzung gespart werden. Allerdings garantieren die Übersetzungsbüros im Gegensatz zu den Online-Diensten eine angemessene Qualität.

Für nicht kommerzielle Zwecke, also beispielsweise private Projekte, darf jedoch die potenziell eingeschränkte, aber kostenlose API der DeepL-Server, oder ähnlicher Anbieter verwendet werden. In privaten Projekten sollte auch die Qualität der Übersetzungen, die hier geboten werden ausreichen.

7 Fazit

Insgesamt ist das Projekt „Erstellung eines Datentreibers zur Verwaltung von Texten in verschiedenen Sprachen“ als Erfolg zu bewerten. Der Datentreiber ist voll funktionsfähig und getestet. Zusätzlich ist die Integration in das AddIn „FxA_Hand“ gelungen. Somit wurden die wichtigsten Requirements erfüllt.

Die Erweiterung des AddIns „FxA_ProgManager“ wurde verschoben. Sobald der ProgManager jedoch das geplante Refactoring erhalten hat, wird die Integration der Mehrsprachigkeit vermutlich trivial sein. Ähnlich wie beim AddIn „FxA_Hand“, wo das erwähnte „Plug-and-Play“ auf Kapitel 3.3.2 bereits getestet wurde.

Literatur

FPT Robotik GmbH (2024). *Whitepaper: Roboterneutralität*.

Microsoft Corporation (2023). *C# preprocessor directives*. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/preprocessor-directives#conditional-compilation>.

— (2021). *Microsoft Data Access Objects reference*. URL: <https://learn.microsoft.com/en-us/office/client-developer/access/desktop-database-reference/microsoft-data-access-objects-reference>.

Harp, Vicky und Diego Vega (2019). *Introducing the new Microsoft.Data.SqlClient*. URL: <https://devblogs.microsoft.com/dotnet/introducing-the-new-microsoftdatasqlclient/#what-does-this-mean-for-system.data.sqlclient2>.

A Anhang

Anhang.zip

└─ Quellen

└─ fpt_Whitepaper_Roboterneutralitaet.pdf

└─ CSharp_Preprocessor-directives.htm

└─ Microsoft_Data_Access_Objects_reference.htm

└─ Introducing_the_new_Microsoft.Data.SqlClient.htm